

C语言输入输出速查手册

覆盖大学 C 语言课程中所有常见的输入输出场景，适合期末复习。

目录

- 1. 输出函数总览
- 2. printf 格式说明符
- 3. printf 进阶格式控制
- 4. 输入函数总览
- 5. scanf 格式说明符
- 6. scanf 的常见陷阱
- 7. 字符输入输出
- 8. 字符串输入输出
- 9. 标准易错点汇总
- 10. 期末考试常见题型

1. 输出函数总览

函数	头文件	用途	示例
printf	<stdio.h>	格式化输出	printf("a=%d", a);
putchar	<stdio.h>	输出单个字符	putchar('A');
puts	<stdio.h>	输出字符串并自动换行	puts("Hello");

2. printf 格式说明符

格式：%[标志][宽度][.精度]类型字符

2.1 基本类型说明符

说明符	含义	示例	输出
%d	有符号十进制整数	printf("%d", 97);	97
%u	无符号十进制整数	printf("%u", -1);	4294967295
%ld	长整型	printf("%ld", 100000L);	100000
%f	单双精度浮点数（小数形式）	printf("%f", 3.14);	3.140000
%lf	双精度浮点数（printf中与%f等价）	printf("%lf", 3.14);	3.140000
%e	科学计数法（小写e）	printf("%e", 314.0);	3.140000e+02
%E	科学计数法（大写E）	printf("%E", 314.0);	3.140000E+02

说明符	含义	示例	输出
%g	自动选%f或%e（不输出多余零）	printf("%g", 3.14);	3.14
%c	单个字符	printf("%c", 65);	A
%s	字符串	printf("%s", "abc");	abc
%o	无符号八进制	printf("%o", 10);	12
%x	无符号十六进制（小写）	printf("%x", 255);	ff
%X	无符号十六进制（大写）	printf("%X", 255);	FF
%p	指针（地址）	printf("%p", &a);	0061FF1C
%%	输出百分号本身	printf("100%%");	100%

2.2 记忆口诀

d 整 f 浮 c 字符 s 串
o 八 x 十六 e 科学
u 无符 ld 长整 % 本身

3. printf 进阶格式控制

3.1 宽度控制

格式	含义	示例 printf	输出
%5d	至少占5列，右对齐	printf("%5d", 42);	42
%-5d	至少占5列，左对齐	printf("%-5d", 42);	42
%05d	不足5列用0填充	printf("%05d", 42);	00042

3.2 精度控制（浮点数）

格式	含义	示例	输出
%.2f	保留2位小数	printf("%.2f", 3.14159);	3.14
%.0f	不保留小数	printf("%.0f", 3.6);	4（四舍五入）
%8.2f	总宽8列，2位小数	printf("%8.2f", 3.14);	3.14
%10.3f	总宽10列，3位小数	printf("%10.3f", 1.5);	1.500

3.3 字符串截断

格式	含义	示例	输出
<code>%.3s</code>	只输出前3个字符	<code>printf("%.3s", "Hello");</code>	Hel
<code>%8.3s</code>	8列宽，只显示前3字符	<code>printf("%8.3s", "Hello");</code>	Hel

4. 输入函数总览

函数	头文件	用途	示例
<code>scanf</code>	<code><stdio.h></code>	格式化输入	<code>scanf("%d", &a);</code>
<code>getchar</code>	<code><stdio.h></code>	读取单个字符	<code>c = getchar();</code>
<code>gets</code>	<code><stdio.h></code>	读取一行字符串（⚠️ 已过时，不安全）	<code>gets(str);</code>
<code>fgets</code>	<code><stdio.h></code>	安全地读取一行字符串	<code>fgets(str, size, stdin);</code>

5. scanf 格式说明符

5.1 基本用法

```
scanf("格式串", &变量1, &变量2, ...);  
//           ↑ 注意：必须加取地址符 &
```

说明符	含义	示例
<code>%d</code>	读入十进制整数	<code>scanf("%d", &a);</code>
<code>%ld</code>	读入长整型	<code>scanf("%ld", &a);</code>
<code>%f</code>	读入 float 浮点数	<code>scanf("%f", &x);</code>
<code>%lf</code>	读入 double 浮点数	<code>scanf("%lf", &x);</code>
<code>%c</code>	读入一个字符（⚠️ 含空格）	<code>scanf("%c", &c);</code>
<code>%s</code>	读入字符串（到空白为止）	<code>scanf("%s", str);</code>

5.2 格式串中的普通字符

```
scanf("a=%d,b=%d", &a, &b);  
//      ^^  ^^  这些普通字符必须原样输入！  
//  正确输入：a=3,b=7  
//  错误输入：3 7  ← 缺少 a= 和 ,
```

5.3 连续输入

```
// 数值连续输入 - 空格/回车/Tab 自动分割
scanf("%d%d%d", &a, &b, &c); // 输入: 3 7 5 或 3换行7换行5

// 字符连续输入 - 无空白分割
scanf("%c%c", &c1, &c2); // 输入: AB (中间不能有空格!)
```

6. scanf 的常见陷阱

6.1 陷阱一: %c 不跳过空白字符

这是考试最高频的坑!

```
scanf("%d", &n);
scanf("%c", &ch);
// 输入: 123换行A
// 结果: n=123, ch='\n' (读到了换行符, 不是'A'!)
```

解决方案:

```
// 方案1: 在 %c 前加空格, 让 scanf 跳过所有空白
scanf(" %c", &ch); // 注意 %c 前面有个空格!

// 方案2: 用 getchar() 吃掉缓冲区残留
scanf("%d", &n);
getchar(); // 吃掉换行符
scanf("%c", &ch);
```

6.2 陷阱二: 格式串中的普通字符必须原样输入

```
scanf("x=%f,y=%f", &x, &y);
// 必须输入: x=8.5,y=71.82 ← 字母和逗号一个不能少
```

6.3 陷阱三: scanf 到 %s 不读空格

```
char str[20];
scanf("%s", str);
// 输入: Hello World
// 结果: str = "Hello" (World 留在缓冲区)
```

6.4 陷阱四: 浮点类型混淆

```
float x;    scanf("%f", &x);    // 正确：用 %f
double y;   scanf("%lf", &y);   // 正确：用 %lf ← 区分！
```

```
// 但 printf 中 %f 和 %lf 通用
printf("%f", x);    // 正确
printf("%f", y);    // 正确（printf对float/double都用%f）
```

7. 字符输入输出

7.1 putchar

```
putchar('A');    // 输出：A
putchar(65);     // 输出：A （ASCII码65='A'）
putchar('\n');   // 输出换行
putchar(c1);     // 输出变量 c1 对应的字符
```

7.2 getchar

```
char c;
c = getchar();   // 从键盘读入一个字符（含空格/换行）
```

典型用途：吃换行符

```
scanf("%d", &n);
getchar();       // 吃掉 scanf 遗留的换行符
c = getchar();   // 现在读到的是真正的字符
```

7.3 字符与 ASCII 码互转

```
char c = 'A';
printf("%c", c);    // 按字符输出 → A
printf("%d", c);    // 按ASCII码输出 → 65

int n = 97;
printf("%c", n);    // 按字符输出 → a
printf("%d", n);    // 按整数输出 → 97
```

8. 字符串输入输出

8.1 puts — 输出字符串

```
char str[] = "Hello";
puts(str);           // 输出 Hello 并自动换行
puts("World");       // 输出 World 并自动换行
```

8.2 gets — 读取一行（✗ 考试可能还在考，但已淘汰）

```
char str[50];
gets(str);           // 读到换行符，丢弃换行，\0 结尾
// ⚠️ 危险：不检查数组长度，可能越界！
```

8.3 fgets — 安全替代

```
fgets(str, 50, stdin); // 最多读49字符，保留\0位置
// ⚠️ 注意：fgets 会保留换行符 \n 在字符串末尾
```

9. 标准易错点汇总

易错点	错误写法	正确写法
scanf 缺 &	scanf("%d", a);	scanf("%d", &a);
printf 加 &	printf("%d", &a);	printf("%d", a);
%c 吃空格	scanf("%c%c", &c1, &c2); 输入 A B	scanf(" %c %c", &c1, &c2);
浮点输入用错	scanf("%d", &x);	scanf("%f", &x);
printf 精度误用	printf("%.2d", 5);	printf("%.2f", 3.14); (精度只对浮点数有意义)
double 输入	scanf("%f", &y);	scanf("%lf", &y);
格式串有普通字符	只输入数据不输入普通字符	必须原样输入格式串中的普通字符

10. 期末考试常见题型

题型一：写输出结果

```
int a = 5, b = 7;
printf("a=%d,b=%d\n", a, b);
// 输出: a=5,b=7
```

题型二：字符与ASCII互转

```
char c = 'A';
printf("%c %d\n", c, c);
// 输出: A 65
```

题型三：格式控制

```
printf("%5d\n", 123);    // 输出: 123
printf("%05d\n", 123);   // 输出: 00123
printf("%.2f\n", 3.1);   // 输出: 3.10
printf("%-5d\n", 123);   // 输出: 123
```

题型四：scanf 陷阱分析

```
int n; char c;
scanf("%d", &n);
scanf("%c", &c);
// 输入: 10换行A
// 问: c 读到了什么?
// 答: '\n' (换行符), 不是 'A'
```

题型五：格式串中的普通字符

```
scanf("a=%db=%d", &a, &b);
// 应如何输入使 a=3, b=7?
// 答: a=3b=7
```

题型六：char 溢出

```
char c = 197;           // 超出 signed char 范围
printf("%d\n", c);
// 输出: -59 (发生溢出)
```

题型七：short-circuit (短路求值)

```
int a = 3, b = 4;
// !(a > b) && !(c = 5) && 0
// 问：这一步执行后 c 的值是？
// 答：c 未被赋值，因为 && 短路 — 左边为假，右边不执行
```

11. 数组的各种定义方式

11.1 一维数组定义

```
// 方式1：定义后再赋值
int a[10];
a[0] = 1; a[1] = 2; ...

// 方式2：全部初始化
int a[5] = {1, 2, 3, 4, 5};

// 方式3：部分初始化（其余自动为 0）
int a[5] = {1, 2};           // {1, 2, 0, 0, 0}

// 方式4：不指定长度（由初始值个数决定）
int a[] = {1, 2, 3, 4, 5};   // 长度为 5

// 方式5：全部初始化为 0
int a[10] = {0};             // {0, 0, 0, 0, 0, 0, 0, 0, 0, 0}

// 方式6：用 #define 定义长度
#define N 10
int a[N];
```

写法	结果
<code>int a[10];</code>	未初始化，值随机
<code>int a[5] = {1,2,3,4,5};</code>	全部指定
<code>int a[5] = {1,2};</code>	前2个赋值，后3个为0
<code>int a[] = {1,2,3};</code>	自动长度为3
<code>int a[10] = {0};</code>	全为0

11.2 二维数组定义

```
// 方式1：按行初始化（推荐）
int a[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};
```

```

    {9,10,11,12}
};

// 方式2: 按顺序初始化 (自动按行存放)
int a[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12}; // 效果同上

// 方式3: 部分初始化
int a[3][4] = {{1}, {5}, {9}};
// 结果: 每行只有第一个有值, 其余为0

// 方式4: 省略行数 (列数不能省!)
int a[][4] = {{1,2,3,4}, {5,6,7,8}}; // ✅ 自动为2行
// int a[3][] = {...}; // ❌ 列数不能省略!

```

⚠️ 二维数组列数必须指定: `int a[][4]` 正确, `int a[3][]` 错误!

12. 字符串的各种定义方式 (★高频考点)

12.1 六种定义方式

```

// 方式1: 字符数组逐个赋值
char str[6] = {'H', 'e', 'l', 'l', 'o', '\0'};

// 方式2: 字符串字面量初始化 (自动加 \0)
char str[6] = "Hello"; // 长度 = 5个字符 + 1个\0 = 6

// 方式3: 不指定长度
char str[] = "Hello"; // 自动分配 6 字节 (含 \0)

// 方式4: 先定义后赋值 (❌ 不能直接用 =)
char str[20];
// str = "Hello"; // ❌ 错误! 数组名是地址常量
strcpy(str, "Hello"); // ✅ 正确, 用 strcpy

// 方式5: 字符指针 (指向常量区, 不可修改)
char *p = "Hello"; // p 指向只读常量区
// p[0] = 'h'; // ❌ 运行时错误!

// 方式6: 动态分配 (考试较少涉及)
char *p = (char*)malloc(20 * sizeof(char));
strcpy(p, "Hello");
free(p);

```

12.2 字符数组 vs 字符指针 (必考对比)

对比项	<code>char s[] = "Hello";</code>	<code>char *p = "Hello";</code>
存储位置	栈 (可读写)	常量区 (只读)

对比项	char s[] = "Hello"; char *p = "Hello";	
能否修改内容	✅ s[0] = 'h';	❌ p[0] = 'h'; 崩溃
能否重新赋值	❌ s = "abc"; 错	✅ p = "abc"; 正确
sizeof 结果	6（数组实际长度）	4 或 8（指针大小）
本质	数组（地址常量）	指针变量

```
char s[] = "Hello";
char *p = "Hello";

s[0] = 'h';      // ✅ 改数组内容，OK
p[0] = 'h';      // ❌ 改常量区，运行时崩溃！

s = "World";     // ❌ 数组名是常量，不能赋值
p = "World";     // ✅ 指针重新指向，OK

sizeof(s);       // 6（整个数组）
sizeof(p);       // 4|8（指针本身大小）
```

13. 数组/字符串输入方式对比

13.1 数值数组输入

```
// 方式1: scanf 逐个读入
for (i = 0; i < n; i++)
    scanf("%d", &a[i]);

// 方式2: 初始化时给定
int a[5] = {1, 2, 3, 4, 5};

// 方式3: scanf 连续读入（空格/换行自动分割）
scanf("%d%d%d", &a[0], &a[1], &a[2]);
```

13.2 字符串输入 — 四种方式对比（★重点）

函数	遇空格	遇换行	自动加\0	越界检查	考试地位
scanf("%s", s)	停止	停止	✅	❌ 不检查	必考
gets(s)	继续读	停止（丢弃\n）	✅	❌ 不检查	常考（已淘汰）
fgets(s, n, stdin)	继续读	停止（保留\n）	✅	✅	推荐替代
getchar() 循环	自己控制	自己控制	手动	自己控制	偶尔考

```
// 示例对比 - 输入 "Hello World\n"
char s1[20], s2[20], s3[20];

scanf("%s", s1);           // s1 = "Hello"           (World没读)
gets(s2);                  // s2 = "Hello World"      (无换行符)
fgets(s3, 20, stdin);      // s3 = "Hello World\n"  (保留换行符!)
```

13.3 吃换行符的几种写法

```
// 场景: scanf 数值后读字符串
scanf("%d", &n);

// 方法1: 在 %s/%c 前加空格
scanf(" %s", str);        // 空格让 scanf 跳过所有空白

// 方法2: getchar 吃掉
getchar();                // 吃掉换行符
gets(str);

// 方法3: fflush (不推荐, 标准只对输出流有效)
fflush(stdin);            // 某些编译器可用, 不通用
```

14. 数组/字符串输出方式对比

14.1 数值数组输出

```
// 方式1: 逐个输出
for (i = 0; i < n; i++)
    printf("%d ", a[i]);

// 方式2: 指定格式对齐
for (i = 0; i < n; i++)
    printf("%5d", a[i]);    // 每个占5列
```

14.2 字符串输出 — 三种方式对比

函数	自动换行	结束条件	适用场景
printf("%s", s)	✗ 不换行	遇 \0 停止	通用, 可加格式控制
puts(s)	✓ 自动换行	遇 \0 停止	简单输出字符串
putchar(s[i]) 循环	✗	自己控制	需要逐字符处理时

```
char s[] = "Hello";
printf("%s\n", s);    // 输出 Hello 换行
puts(s);              // 输出 Hello 换行（效果同上）
printf("%.3s", s);    // 输出 Hel（截断，puts 做不到）
```

15. 各函数串讲总结

C 语言 I/O 全家桶		
printf	输出	格式化输出，万能
scanf	输入	格式化输入，记加 &
putchar	输出	输出单个字符
getchar	输入	读单个字符，常用来吃换行
puts	输出	输出字符串，自动换行
gets	输入	读一行（ ✗ 越界风险，已淘汰）
fgets	输入	安全读一行（保留\n）
strlen	工具	求字符串长度（不计\0）
strcpy	工具	拷贝字符串（含\0）
strcat	工具	拼接字符串
strcmp	工具	比较字符串

函数	需要 &?	遇空格	遇 \0	自动换行
printf	✗	—	—	✗
scanf("%d")	✓	跳过	—	—
scanf("%s")	✗	停	—	—
scanf("%c")	✓	不跳过	—	—
gets	✗	继续	—	✗ （丢弃\n）
fgets	✗	继续	—	✗ （保留\n）
puts	✗	—	停	✓
putchar	✗	—	—	✗

最后提醒：考试中 scanf 扣分最多的就是忘记 & 和 %c 吃空格/换行，务必注意！